

01

SESSION 4

Strings, Number Checking, Conversion & DOM Fundamentals

JS

JavaScript Notes

Session 4

Strings → Number checks → Conversion → DOM

Notebook goal

Work with text, check numeric values, convert input safely, and read data from HTML using the DOM.

Reference basis: the supplied Session 4 JS, HTML and PDF. The wording and examples here are rewritten in simple language.

Table of Contents

Clickable index - jump to a topic and use Return to Index on every page

03 Session 4 Roadmap

04 String Methods Overview

05 length

06 toLowerCase() / toUpperCase()

07 split()

08 indexOf() / lastIndexOf()

09 slice()

10 match()

11 concat()

12 replace()

13 replaceAll()

14 startsWith() / endsWith()

15 charAt()

16 isWellFormed()

17 Related String Methods I

18 Related String Methods II

19 Related String Methods III

20 String Method Pattern + Immutability

21 Number Checking

22 isNaN()

23 Number.isNaN() / isFinite() / isInteger() / isSafeInteger

24 Number Conversion Overview

25 Number()

26 parseInt()

27 parseFloat()

28 Number Conversion Comparison

29 DOM Basics - getElementById()

30 getElementsByTagName()

31 Related DOM Selectors

32 Collections + Duplicate IDs

33 getData() Walkthrough

34 HTML -> JavaScript Data Flow

35 Practice 1-10

36 Practice 11-20

37 Answer Key 1-10

38 Answer Key 11-20

39 Mini Project - Input Validator

40 Interview Revision

41 Quick Revision Sheet

42 Session Summary

43 Clean Source Code - Part 1

44 Clean Source Code - Part 2

45 Clean Source Code - Part 3

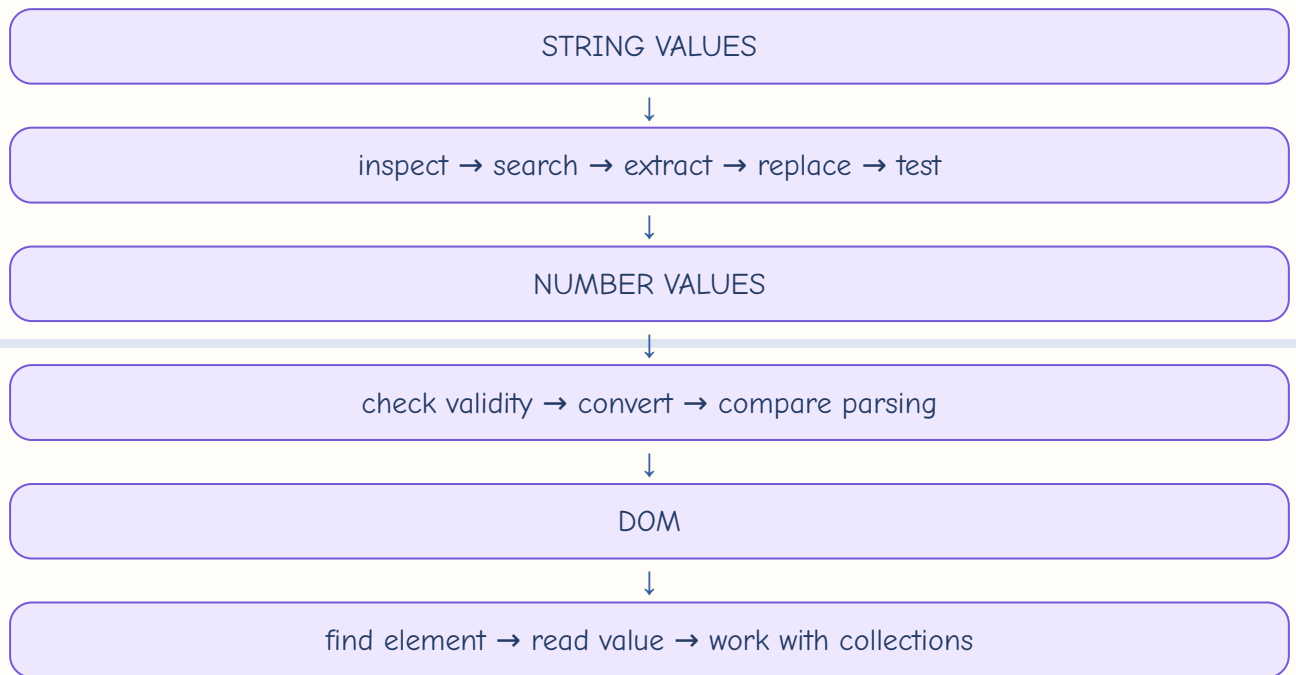
46 Clean Source HTML

Navigation tip

Click a topic. At the bottom of each note page, use Return to Index to come back here.

Session 4 Roadmap

How the ideas connect



Core idea

Session 4 connects data handling with the browser. A string is manipulated as text, a number is checked/converted, and the DOM lets JavaScript read data from HTML.

String Methods Overview

A practical toolkit for working with text

```
let str = "I am learning Javascript learning";
```

String methods help you inspect, search, extract, transform and format text. Most methods return a new value instead of rewriting the original string.

Group	Methods
Inspect	length, charAt, at
Search	indexOf, lastIndexOf, includes, search, match
Extract	slice, substring
Transform	toLowerCase, toUpperCase, replace, replaceAll
Combine	concat
Test	startsWith, endsWith, isWellFormed
Format	trim, repeat, padStart, padEnd

Remember

Think: original text → method call → returned result. Store the returned result when you need it later.

length

String methods

Gives the size of the string. Spaces count too.

```
str.length
```

Example

```
const text = "JavaScript is fun";  
console.log(text.length);
```

What happens

You get a number. For ordinary English text this behaves like a character count.

Remember

length is a property, not a function. Write `text.length`, not `text.length()`.

toLowerCase() / toUpperCase()

String methods

Makes a lowercase or uppercase version.

```
str.toLowerCase();  
str.toUpperCase();
```

Example

```
const text = "JavaScript";  
console.log(text.toLowerCase());  
console.log(text.toUpperCase());
```

What happens

```
javascript  
JAVASCRIPT
```

Remember

The original string stays unchanged.

split()

String methods

Turns one string into an array using a separator.

```
str.split(separator)
```

Example

```
const line = "red,green,blue";  
console.log(line.split(","));
```

What happens

```
["red", "green", "blue"]
```

Remember

With no separator, the whole string becomes one array item.

Common pitfall

`split("")` works at UTF-16 code-unit level, so it is not always the same as user-perceived characters.

indexOf() / lastIndexOf()

String methods

Finds the first or last matching position.

```
str.indexOf(value);  
str.lastIndexOf(value);
```

Example

```
const text = "banana";  
console.log(text.indexOf("a"));  
console.log(text.lastIndexOf("a"));
```

What happens

1
5

Remember

If the value is not found, the result is -1.

Common pitfall

Do not use `if (text.indexOf("a"))` to test existence because index 0 is a valid match. Use `!== -1`.

slice()

String methods

Extracts part of a string. The end index is excluded.

```
str.slice(start, end)
```

Example

```
const text = "JavaScript";  
console.log(text.slice(0, 4));  
console.log(text.slice(-3));
```

What happens

Java
ipt

Remember

Negative indexes count from the end.

Common pitfall

slice(0, 4) gives indexes 0,1,2,3. The 4 is not included.

match()

String methods

Finds matches using a regular expression.

```
str.match(pattern)
```

Example

```
const text = "learn learn";  
console.log(text.match(/learn/g));
```

What happens

```
["learn", "learn"]
```

Remember

The g flag means global matching. No match can produce null.

concat()

String methods

Joins strings and returns the combined string.

```
str.concat(value1, value2, ...)
```

Example

```
const first = "Hello";  
console.log(first.concat(" JavaScript"));
```

What happens

Hello JavaScript

Remember

For simple joining, the + operator is also common.

replace()

String methods

Replaces the first matching occurrence when the search value is a string.

```
str.replace(searchValue, newValue)
```

Example

```
const text = "hi hi";  
console.log(text.replace("hi", "hello"));
```

What happens

hello hi

Remember

Use `replaceAll()` or a global pattern when every occurrence should change.

replaceAll()

String methods

Replaces every matching string occurrence.

```
str.replaceAll(searchValue, newValue)
```

Example

```
const text = "hi hi";  
console.log(text.replaceAll("hi", "hello"));
```

What happens

hello hello

Remember

replace() and replaceAll() answer different questions: first occurrence vs all occurrences.

startsWith() / endsWith()

String methods

Checks the beginning or ending of a string.

```
str.startsWith(value);  
str.endsWith(value);
```

Example

```
const file = "notes.pdf";  
console.log(file.startsWith("notes"));  
console.log(file.endsWith(".pdf"));
```

What happens

```
true  
true
```

Remember

Both return true or false and are case-sensitive.

charAt()

String methods

Reads a character at a zero-based index.

```
str.charAt(index)
```

Example

```
const word = "Code";  
console.log(word.charAt(0));  
console.log(word.charAt(3));
```

What happens

C
e

Remember

Modern alternatives include `word[0]` and `word.at(0)`. Out-of-range `charAt()` returns an empty string.

isWellFormed()

String methods

Checks whether a string has well-formed Unicode code units.

```
str.isWellFormed()
```

Example

```
const text = "Hello";  
console.log(text.isWellFormed());
```

What happens

true

Remember

It is more specialized than the other beginner methods, but it appears in this session.

Related String Methods I

Useful methods that complete the same family

Method	Simple meaning	Example
<code>includes()</code>	Does the text contain this value?	<code>text.includes("JS")</code>
<code>search()</code>	Find the first regex match index	<code>text.search(/JS/)</code>
<code>substring()</code>	Extract using two non-negative indexes	<code>text.substring(0,4)</code>

When to use which

Use `includes()` for a clear yes/no check, `search()` when a regular expression is useful, and `substring()` when you specifically want substring-style index rules.

Related String Methods II

Whitespace, indexing and character information

Method	Use	Example
<code>trim()</code>	Remove whitespace from both ends	<code>input.trim()</code>
<code>trimStart()</code>	Remove leading whitespace	<code>input.trimStart()</code>
<code>trimEnd()</code>	Remove trailing whitespace	<code>input.trimEnd()</code>
<code>at()</code>	Read a character and support negative indexes	<code>text.at(-1)</code>
<code>charCodeAt()</code>	Return the UTF-16 code-unit number	<code>text.charCodeAt(0)</code>

Easy memory trick

`at(-1)` is a convenient way to ask for the last character. `charCodeAt()` gives a number rather than the character itself.

Related String Methods III

Formatting helpers

Method	Use	Example
<code>repeat()</code>	Repeat a pattern	<code>"-".repeat(5)</code>
<code>padStart()</code>	Pad the beginning to a target length	<code>"7".padStart(3,"0")</code>
<code>padEnd()</code>	Pad the end to a target length	<code>"7".padEnd(3,"0")</code>
<code>matchAll()</code>	Walk through all regex matches and capture groups	<code>text.matchAll(/go/g)</code>

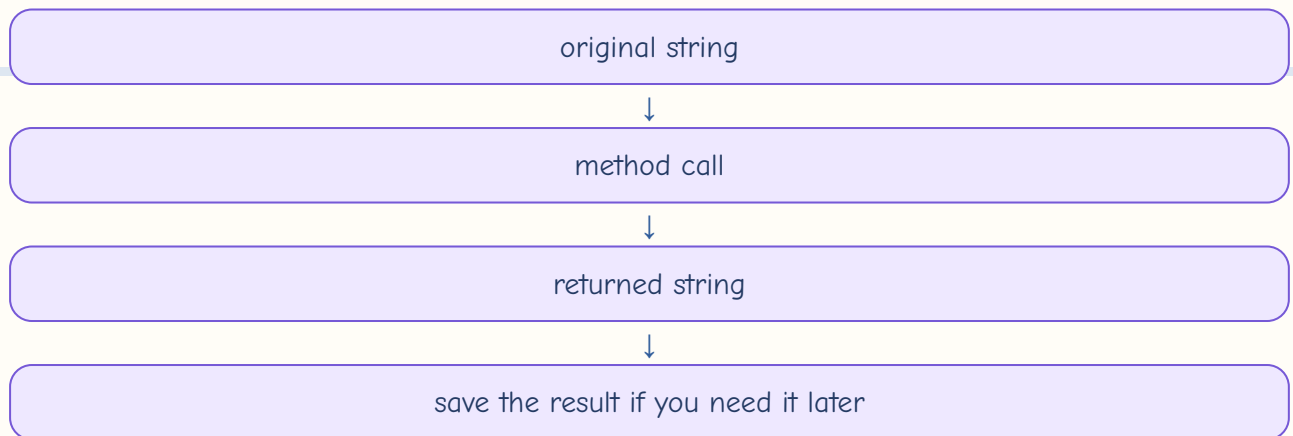
Practical use

These helpers are useful for formatting labels, IDs, reports and repeated text patterns.

String Method Pattern + Immutability

How to think about method calls

```
let text = " hello ";  
let cleaned = text.trim();  
let upper = cleaned.toUpperCase();  
  
console.log(text);  
console.log(upper);
```



Important

Strings are primitive values. String methods return new string values; they do not mutate the original primitive in place.

Number Checking

Global `isNaN()` vs the `Number.is*` family

Tool	Question it answers
<code>isNaN(value)</code>	Does conversion of this value produce NaN?
<code>Number.isNaN(value)</code>	Is this value actually NaN?
<code>Number.isFinite(value)</code>	Is this a finite number?
<code>Number.isInteger(value)</code>	Is this an integer?
<code>Number.isSafeInteger(value)</code>	Is this an integer in the safe range?

Why this matters

Validation should ask a precise question. The `Number.is*` methods avoid the broad coercion behavior of global `isNaN()`.

isNaN()

Global numeric validity check

```
let value = 4;
console.log(isNaN(value));

if (isNaN(value)) {
  console.log("Invalid number");
} else {
  console.log("Valid number");
}
```

Read it

Because 4 is a number, `isNaN(4)` is false and the else block runs.

Caution

The global `isNaN()` converts its argument before checking. That can be surprising for strings.

Number.isNaN(), isFinite(), isInteger(), isSafeInteger()

More explicit checks

```
console.log(Number.isNaN(NaN));  
console.log(Number.isFinite(12));  
console.log(Number.isInteger(12));  
console.log(Number.isSafeInteger(12));
```

Output

```
true  
true  
true  
true
```

Rule of thumb

Prefer Number.is* methods when you want a specific numeric check without broad type coercion.

Number Conversion Overview

The same input string can be interpreted in different ways

```
const value = "12.34abc";
```

```
Number(value);  
parseInt(value, 10);  
parseFloat(value);
```

Function	Simple rule
Number()	The complete value must represent a valid number.
parseInt()	Read an integer prefix.
parseFloat()	Read a decimal-number prefix.

Number()

Whole-value conversion

```
console.log(Number("12.34"));  
console.log(Number("12.34abc"));  
console.log(Number(""));
```

Output

```
12.34  
NaN  
0
```

Simple rule

Use `Number()` when the whole input is supposed to be numeric. Validate the result when input may be invalid.

parseInt()

Read an integer prefix

```
console.log(parseInt("12.34abc", 10));  
console.log(parseInt("101", 2));  
console.log(parseInt("42px", 10));
```

Output

```
12  
5  
42
```

Important

parseInt() stops when the next character cannot belong to the integer. The radix argument is recommended.

parseFloat()

Read a decimal prefix

```
console.log(parseFloat("12.34abc"));  
console.log(parseFloat("42px"));  
console.log(parseFloat("3.14"));
```

Output

```
12.34  
42  
3.14
```

Pair it with parseInt()

parseInt() is for an integer prefix. parseFloat() is for a decimal prefix.

Number Conversion Comparison

Memorize the pattern

Input	Number()	parseInt()	parseFloat()
"12.34abc"	NaN	12	12.34
"42px"	NaN	42	42
"12.34"	12.34	12	12.34
""	0	NaN	NaN

Memory map

Number() = whole value | parseInt() = integer prefix | parseFloat() = decimal prefix

DOM Basics - getElementById()

Select one element by its id

The DOM is the browser representation of the HTML document. JavaScript uses document methods to find elements and read their properties.

```
const input = document.getElementById("name");  
console.log(input.value);
```

Think in two steps

1) Find the element. 2) Read a property such as .value or .textContent.

Important

An id is intended to identify one element in a document.

getElementsByTagName()

Collect elements by tag name

```
const paragraphs = document.getElementsByTagName("p");  
console.log(paragraphs);  
console.log(paragraphs[0].textContent);
```

What you get

An HTMLCollection containing all matching paragraph elements. It can contain zero, one, or many items.

Related DOM Selectors

Natural next steps from id and tag selectors

Method	Purpose	Example
<code>querySelector()</code>	First element matching a CSS selector	<code>document.querySelector("p")</code>
<code>querySelectorAll()</code>	All matching elements as a NodeList	<code>document.querySelectorAll("p")</code>
<code>getElementsByClassName()</code>	Elements sharing a class name	<code>document.getElementsByClassName("item")</code>

Easy rule

`querySelector()` gives the first match. `querySelectorAll()` gives all matches.

Collections + Duplicate IDs

Why the HTML structure matters

```
<div id="para1">1</div>
<div id="para1">2</div>
<p>Lorem ipsum dolor sit.</p>
```

Problem in the supplied HTML

The source uses id="para1" twice. IDs are meant to be unique. Use unique ids, or use a class for repeated elements.

Collection thinking

Tag and class selectors can return collections. An id selector is intended to point to one specific element.

getData() Walkthrough

Read the input and inspect paragraph elements

```
function getData() {  
  const res = document.getElementById("name").value;  
  console.log("res", res);  
  
  const para1 = document.getElementsByTagName("p");  
  console.log("para1 =>", para1);  
}
```

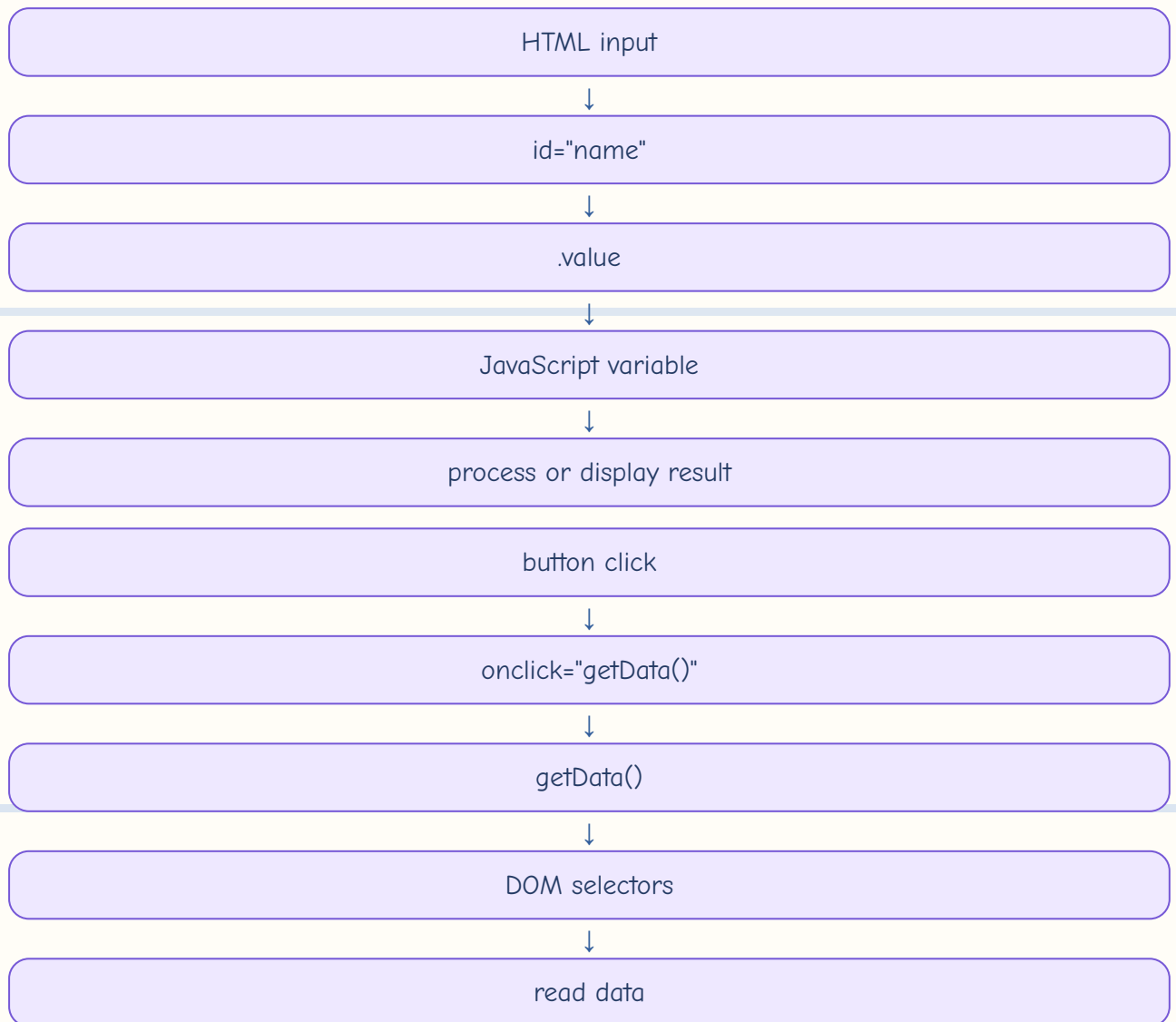
Step	What happens
1	The user types in the input.
2	The button calls getData().
3	getElementById("name") finds the input.
4	.value reads the current text.
5	getElementsByTagName("p") returns the paragraph collection.

HTML connection

The supplied button uses onclick="getData()", so the function runs when the button is clicked.

HTML -> JavaScript Data Flow

The browser interaction chain



Key idea

The DOM is the bridge between the HTML structure and JavaScript logic.

Practice 1-10

Solve without opening the answer key

1. Print the length of a user-provided string.

2. Convert a string to lowercase and uppercase.

3. Check whether the word "learning" exists using includes().

4. Find the first and last index of a chosen character.

5. Extract the last 3 characters using slice().

6. Replace the first occurrence of a word.

7. Replace all occurrences of a word.

8. Check whether a string starts with a chosen prefix.

9. Check whether a string ends with a chosen suffix.

10. Use charAt() and at() to read characters.

Practice 11-20

Number, conversion and DOM exercises

11. Check whether a value is NaN using `isNaN()`.

12. Compare global `isNaN()` with `Number.isNaN()`.

13. Check whether a number is finite and an integer.

14. Convert "123.45abc" using `Number()`, `parseInt()` and `parseFloat()`.

15. Ask for a number and reject invalid input.

16. Read an input element by id and print its value.

17. Read paragraph elements and print the first paragraph text.

18. Use `querySelector()` to select the first paragraph.

19. Use `querySelectorAll()` to count matching paragraphs.

20. Fix the duplicate id issue in the supplied HTML.

Answer Key 1-10

Suggested solutions

```
1. console.log(value.length);
2. console.log(value.toLowerCase(), value.toUpperCase());
3. console.log(value.includes("learning"));
4. console.log(value.indexOf(target), value.lastIndexOf(target));
5. console.log(value.slice(-3));
6. console.log(value.replace(target, replacement));
7. console.log(value.replaceAll(target, replacement));
8. console.log(value.startsWith(prefix));
9. console.log(value.endsWith(suffix));
10. console.log(value.charAt(0), value.at(-1));
```

Learning point

Change the test values and predict the output before running the code.

Answer Key 11-20

Suggested solutions

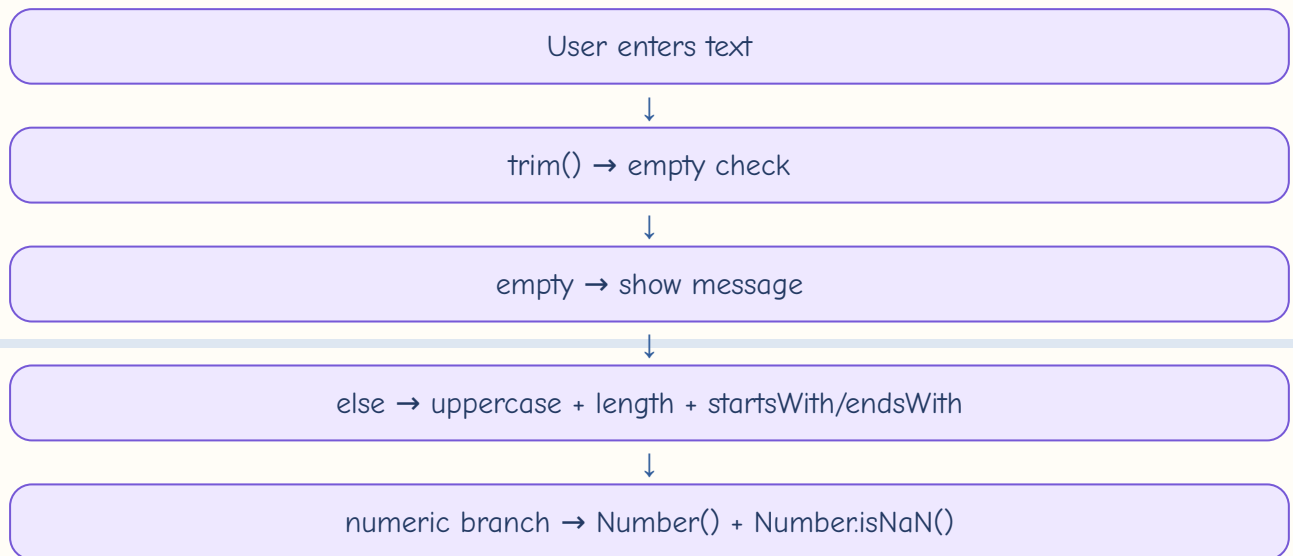
```
11. console.log(isNaN(value));
12. console.log(isNaN("123x"), Number.isNaN("123x"));
13. console.log(Number.isFinite(value), Number.isInteger(value));
14. console.log(Number(s), parseInt(s, 10), parseFloat(s));
15. const n = Number(input); console.log(Number.isNaN(n) ? "Invalid" : "Valid");
16. console.log(document.getElementById("name").value);
17. console.log(document.getElementsByTagName("p")[0].textContent);
18. console.log(document.querySelector("p"));
19. console.log(document.querySelectorAll("p").length);
20. Replace duplicate ids with unique ids or a shared class.
```

DOM reminder

Selection and reading are separate steps: find the element first, then read `.value` or `.textContent`.

Mini Project - Input Validator

Combine the Session 4 ideas



Requirement	Concept
Read input	getElementById() + .value
Clean input	trim()
Text analysis	length, includes(), startsWith(), endsWith()
Text transform	toUpperCase(), replaceAll()
Numeric branch	Number() + Number.isNaN()
Output	console.log() or a DOM update

```

function validateInput() {
  const raw = document.getElementById("name").value;
  const value = raw.trim();

  if (value === "") {
    console.log("Please enter something");
    return;
  }

  console.log("Upper:", value.toUpperCase());
  console.log("Length:", value.length);
  console.log("Starts with J:", value.startsWith("J"));
}
  
```

Why this is useful

It combines DOM reading, string cleanup, string methods and condition-based thinking in one small browser task.

Interview Revision

Fast answers

Question	Answer
What does length return?	String length as UTF-16 code units.
indexOf() vs lastIndexOf()	First matching index vs last matching index.
What does slice() do?	Extracts a portion; end index is excluded.
replace() vs replaceAll()	First occurrence vs all occurrences for a string search.
What does isNaN() do?	Checks whether a value becomes NaN after numeric conversion.
Number.isNaN()	Checks whether the value itself is NaN without broad coercion.
Number() vs parseInt()	Whole-value conversion vs integer-prefix parsing.
getElementById()	Selects the element with a matching id.
getElementsByName()	Returns matching elements as an HTMLCollection.
Why avoid duplicate ids?	An id is intended to be unique in the document.

Quick Revision Sheet

Session 4 at a glance

STRING

length | toLowerCase | toUpperCase | split
indexOf | lastIndexOf | slice | match
concat | replace | replaceAll
startsWith | endsWith | charAt | isWellFormed
includes | search | substring | trim | at
repeat | padStart | padEnd | matchAll

NUMBER CHECKING

isNaN | Number.isNaN | Number.isFinite
Number.isInteger | Number.isSafeInteger

NUMBER CONVERSION

Number() | parseInt() | parseFloat()

DOM

getElementById | getElementsByTagName
querySelector | querySelectorAll | getElementsByClassName

Memory map

String → inspect / search / transform. Numbers → validate / convert. DOM → select / read.

Session 4 Summary

What you should be able to do now

Skill	Expected ability
String inspection	Measure length and read characters
String searching	Find text with indexes, includes or regex methods
String extraction	Use slice and substring
String transformation	Use case conversion and replacement methods
String formatting	Trim, repeat and pad text
Number validation	Use isNaN and Number.is* checks
Number conversion	Choose Number, parseInt or parseFloat for the task
DOM selection	Read elements by id, tag name and CSS selectors
DOM collections	Work with HTMLCollection / NodeList results
Browser input	Connect an HTML input and button to a JavaScript function

Next step

Use these methods inside conditions, loops and small DOM-driven applications.

Clean Source Code - Part 1

Appendix based on the supplied JavaScript

```
// String Methods
let str = "I am learning Javascript learning";

console.log(str.length);
console.log(str.toLowerCase());
console.log(str.toUpperCase());
console.log(str.split());
console.log(str.indexOf("r"));
console.log(str.lastIndexOf("m"));
console.log(str.slice(-1));
console.log(str.match(/learning/g));
console.log(str.concat(", Great"));
console.log(str.replace("learning", "hey"));
console.log(str.replaceAll("learning", "hey"));
console.log(str.startsWith("I"));
console.log(str.endsWith("g"));
console.log(str.charAt(0));
console.log(str.isWellFormed());
```

Clean Source Code - Part 2

Appendix based on the supplied JavaScript

```
// Number Checking
let a = 4;
console.log(isNaN(a));

if (isNaN(a)) {
  console.log("Enter a valid number");
} else {
  console.log("Valid number");
}

// Number Conversion
let num1 = "12.34abc";
console.log(Number(num1));
console.log(parseInt(num1));
```

Clean Source Code - Part 3

DOM function from the supplied JavaScript

```
function getData() {  
  const res = document.getElementById("name").value;  
  console.log("res", res);  
  
  const para1 = document.getElementsByTagName("p");  
  console.log("para1 =>", para1);  
}
```

Clean Source HTML

Appendix based on the supplied HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script src="s4.js"></script>
</head>
<body>
  <input type="text" id="name">
  <button onclick="getData()">Enter</button>
  <div id="para1">1</div>
  <div id="para1">2</div>
  <p>Lorem ipsum dolor sit.</p>
</body>
</html>
```

HTML quality note

The supplied HTML contains duplicate id="para1" values. Keep ids unique, or use a shared class when multiple elements need the same selector.